

모델링 및 평가 수집된 데이터 및 전처리 문서

□ 개요

- 산출물 단계 : 모델링 및 평가
- 평가 산출물 : 수집된 데이터 및 전처리 문서
- 제출 일자 : 2026.03.12
- 깃허브 경로 : <https://github.com/skn21-final-1team>
- 작성 팀원 : 우재현

목차

- 개요
- 데이터 수집
- 전처리 (`crawl/preprocess.py`)
- LLM 정제 및 요약 (`llm/`)
- 청킹 (`chunk/`)
- 임베딩 (`embed/`)
- 벡터 DB 저장
- 데이터 현황

1. 개요

1.1 데이터 설명

본 프로젝트의 데이터는 사용자가 저장한 웹 페이지의 본문(마크다운 텍스트)입니다. 기술 블로그, 공식 문서, 위키, 뉴스 등 다양한 도메인의 웹 페이지를 수집하며, 한국어와 영어 혼합 텍스트로 구성됩니다.

항목	내용
형식	마크다운 텍스트 (헤더, 리스트, 코드블록, 테이블 구조 포함)
언어	한국어/영어 혼합
평균 길이	페이지당 약 1,000~50,000자
저장소	PostgreSQL `source` 테이블

1.2 수집 목적

RAG(Retrieval-Augmented Generation) 시스템의 검색 대상 문서를 구축하는 것이 주 목적입니다. 사용자의 자연어 질문에 대해 관련 문서 청크를 벡터 유사도로 검색하여 LLM의 응답 근거로 제공하기 위함입니다.

1.3 처리 파이프라인

데이터 처리 파이프라인은 다음과 같은 단계를 거칩니다.

URL 입력 → 크롤링 → 전처리 → vLLM 정제/요약 → 청킹 → 임베딩 → PGVector 적재

2. 데이터 수집

2.1 데이터 출처

사용자가 저장한 웹 페이지 URL(블로그, 기술 문서, 위키, 뉴스 등)을 출처로 합니다. 특정 도메인에 제한을 두지 않으며, robots.txt를 준수하여 수집합니다.

2.2 수집 방식 — 하이브리드 크롤링 (`crawl`)

정적 및 동적 페이지를 모두 처리하기 위해 2단계 하이브리드 크롤링 방식을 사용합니다.

단계	도구	조건
1차: 정적 크롤링	`httpx` + `Trafilatura`	HTML 응답에서 본문 추출
2차: 동적 크롤링	`Playwright` (Chromium)	1차 정적 결과가 임계값(기본 300자) 미달 시 fallbacks

- 동적 크롤링 상세: `playwright-stealth`로 봇 탐지를 우회하고, 접힌 콘텐츠(FAQ, 아코디언)를 자동 펼침(`page\_actions.expand\_collapsed`) 처리합니다. JavaScript 렌더링 완료 대기 후 HTML을 파싱하며, 수집 결과는 마크다운 형식의 본문 텍스트로 `source.raw` 컬럼에 저장됩니다.

3. 전처리 (`crawl/preprocess.py`)

3.1 전처리 파이프라인

`MarkdownPreprocessor.run()` 메서드가 다음 단계를 순차적으로 실행합니다.

순서	처리	설명
1	`strip_whitespace`	줄 끝 공백 제거, 앞뒤 공백 정리
2	`remove_html_tags`	잔여 HTML 태그 제거 (``, `` 등)
3	`remove_reference_markers`	위키 스타일 참조 마커 제거 (`[1]`, `[edit]`)
4	`normalize_headings`	헤더 문법 정규화 (`##제목` → `### 제목`)
5	`normalize_list_markers`	리스트 마커 통일 (`*`, `+` → `-`)
6	`repair_code_fences`	짜이 맞지 않는 코드 펜스 복구
7	`normalize_table_separators`	마크다운 테이블 구분선 정규화
8	`normalize_blank_lines`	연속 3줄 이상 빈 줄을 2줄로 축소
9	`strip_footer`	저작권/푸터 문구 제거

3.2 설계 원칙

마크다운 구조(``#``, ``\n``, 리스트, 코드블록)는 이후 청킹에서 분할 기준으로 활용하기 위해 보존하며, 본문 내용 자체는 변경하지 않고 노이즈만 제거하는 것을 원칙으로 합니다.

4. LLM 정제 및 요약 (`llm/`)

4.1 정제 (Refine)

전처리된 마크다운을 LLM에 통과시켜 크롤링 잔여 노이즈를 제거합니다.

항목	내용
모델	Meta Llama 3.1 8B Instruct
인프라	RunPod Serverless (`/run` + polling)
입력	전처리된 마크다운 (최대 50,000자, 초과 시 절삭)
출력	노이즈 제거된 마크다운 (원본 90% 이상 유지)

- 정제 규칙 (프롬프트): 스크립트 잔여물(``javascript:void``), UI/네비게이션 텍스트(``좋아요``, ``공유``, ``로그인``), 파편화된 문자(``한\n화\n시\n스\n템``), 중복 문장/반복 패턴 등을 제거하며, 문단 구조와 줄바꿈은 원본을 유지합니다.

```

_REFINE_TEMPLATE = """\
<|begin_of_text|><|start_header_id|>system<|end_header_id|>
당신은 웹 문서 노이즈 정제 전문가입니다.
사용자가 제공하는 웹 문서 본문에서 노이즈만 제거하고, 정제된 본문을 반환하세요.

[정제 규칙 - 최우선: 축약 금지]
- 원본 문서의 모든 문장, 문단, 데이터를 그대로 유지하세요.
- 정제 후 글자 수는 반드시 원본의 90% 이상이어야 합니다.
- 제거 대상은 오직 다음 항목뿐입니다:
  * 크롤링 과정에서 섞인 스크립트 잔여물 (예: javascript:void, [object Object])
  * UI/네비게이션 텍스트 (예: 좋아요, 공유, 로그인, 메뉴, 이전/다음)
  * 파편화된 단어나 깨진 문자 (예: 한 글자씩 떨어진 "한\\n화\\n시\\n스\\n템")
  * 중복된 문장이나 반복 패턴
  * 푸터/저작권 문구 (예: "모든 콘텐츠는 저작권법의 보호를 받습니다", "무단 전재 및 재배포 금지", "Copyright ©")
- 위 항목 외에는 단어 하나도 삭제하거나 바꾸지 마세요.
- 문단 구조와 줄바꿈을 원본 그대로 유지하세요.
- "정제된 본문은 다음과 같습니다" 같은 설명을 절대 붙이지 마세요.

반드시 마크다운 코드 블록 없이 순수한 JSON 형식으로만 응답하세요:
{{"refined": "정제된 전체 본문"}}
<|eot_id|><|start_header_id|>user<|end_header_id|>
다음 웹 문서 본문을 정제해주세요. 절대 축약하지 말고 원본 길이를 유지하세요:

```

4.2 요약 (Summarize)

정제된 본문으로부터 페이지 주제 소개 문구를 4~5문장으로 생성합니다. 이는 검색 결과 미리보기 및 문서 분류 보조 용도로 활용되며, 본문 언어에 맞춰 생성됩니다.

4.3 저장

전처리 산출물은 다음 컬럼에 저장됩니다.

컬럼	내용
`source.raw`	크롤링 원본 마크다운
`source.refined`	LLM 정제된 마크다운
`source.summary`	페이지 소개 요약

5. 청킹 (`chunk/`)

5.1 실험한 청킹 전략 (10종)

MarkdownHeader, Semantic, Recursive 등 다양한 전략으로 불특정 웹 페이지 들에 공통적으로 성능이 좋은 전략을 찾는 실험을 진행했습니다.

ID	전략	방식	특징
----	----	----	----

S1	Recursive	`RecursiveCharacterTextSplitter`	문자 수 기반 재귀 분할, 베이스라인
S2	Token	`TokenTextSplitter` (tiktoken)	토큰 수 기반 분할
S3	Semantic	`SemanticChunker`	임베딩 유사도 기반 의미 단위 분할
S4	MarkdownHeader	`MarkdownHeaderTextSplitter`	H1/H2/H3 헤더 기준 분할
S5	Markdown	`MarkdownTextSplitter`	마크다운 구문 인식 분할
S5-P	Markdown + Prepend	S5 + 헤더 경로 주입	`[# A > ## B] 본문...` 형태
S6	Hierarchical	MarkdownHeader → Recursive 재분할	헤더 분할 후 크기 초과 시 재분할
S6-P	Hierarchical + Prepend	S6 + 헤더 경로 주입	계층 구조 맥락 보존
S7	Contextual Retrieval	S5 + GPT-4o 맥락 요약 prepend	문서 전체 맥락을 청크에 주입
S8	Sentence Window	문장 단위 분할 + window 확장	전후 문장 포함으로 맥락 보존

5.2 실험한 청크 크기 (7종)

불특정 웹페이지에 대해 공통적으로 좋은 성능을 위해 청크 크기와 중첩 크기를 다양하게 실험했습니다.

chunk_size	chunk_overlap	overlap 비율
256 ▾	30 ▾	11.7% ▾
500 ▾	50 ▾	10.0% ▾
800 ▾	100 ▾	12.5% ▾
1,000 ▾	100 ▾	10.0% ▾

1,000 ▾	200 ▾	20.0% ▾
1,500 ▾	150 ▾	10.0% ▾
2,000 ▾	200 ▾	10.0% ▾

5.3 청킹 품질 평가 지표

지표	설명
paragraph	단락 경계 보존율
sentence	문장 무결성 (경계에서 깨지지 않는 비율)
heading	헤더가 청크 시작부에 위치하는 비율
list	리스트 항목 보존율
boundary	자연스러운 위치에서 끊기는 비율
coverage	청크 총 길이 / 원본 길이 (1.0 최적)

5.4 청킹 분석 결과 (상위 5개)

10개 전략 x 7개 크기 = 57개 조합을 평가한 결과, S7(Contextual Retrieval) 전략이 경계 품질 100%로 상위권을 독점했습니다. GPT-4o 모델로 각 청크에 문서 맥락을 주입하여 청킹을 진행한 “S7 Contextual” 전략이 가장 검색 정확도가 높으나 한번에 많은 요청처리에 대해 시간소요가 급증하는 경향을 보였습니다. 때문에 우선적으로는 그 다음으로 성능을 보인 S3(Semantic), S5(MD구분자) 전략을 사용하는 방향으로 계획을 정했으며, 성능미달 시 S7((Contextual Retrieval)은 모델 성능을 낮추거나 개선하여 사용할 예정입니다.

순위	전략 / 크기	구조 보존	커버리지	경계 품질
1	S7 Contextual / 2000_200	65.9%	114.3%	100.0% ▾
2	S7 Contextual / 1500_150	67.1%	118.2%	100.0% ▾
3	S3 Semantic / auto	67.8%	99.7%	77.0% ▾
4	S7 Contextual / 1000_100	66.8%	122.9%	100.0% ▾
5	S7 Contextual / 800_100	66.5%	130.4%	100.0% ▾

6. 임베딩 (`embed/`)

6.1 임베딩 모델

항목	내용
모델	`BAAI/bge-m3`
차원	1,024
인프라	RunPod Serverless (`/runsync`)
특징	다국어 지원 (한국어/영어 혼합 문서에 적합)

6.2 임베딩 품질 평가 지표

지표	설명	기준
cos 평균	전체 벡터 간 코사인 유사도 평균	낮을수록 변별력 높음
cos std	유사도 분포 표준편차	높을수록 변별력 있음
Isotropy	벡터 공간 활용 균일도	1.0에 가까울수록 고르게 분포
Silhouette	source_id별 클러스터링 품질	1.0에 가까울수록 같은 문서 청크끼리 응집
중복 비율	코사인 유사도 ≥ 0.95 인 쌍의 비율	낮을수록 좋음

6.3 임베딩 분석 결과 (상위 5개)

57개 조합에 대해 벡터 품질을 평가한 결과, S7(Contextual Retrieval) 전략이 Silhouette 0.44로 다른 전략 대비 월등했습니다. 맥락 요약이 같은 문서 내 청크끼리의 의미적 응집도를 높인 것으로 분석됩니다. 뒤를 이어서 S2, S1 전략 등이 순위에 매겨졌으며 이를 고려해서 retriever 테스트 순서에 적용할 계획입니다.

순위	전략 / 크기	cos 평균	Silhouette	중복 비율
1	S7 Contextual / 800_100	0.4263	0.4418	0.008% ▾
2	S7 Contextual / 1000_200	0.4314	0.4391	0.0% ▾

3	S7 Contextual / 1000_100	0.4280	0.4325	0.0% ▾
4	S7 Contextual / 500_50	0.4211	0.4112	0.001% ▾
5	S7 Contextual / 1500_150	0.4354	0.4222	0.0% ▾

7. 벡터 DB 저장

7.1 저장 구조

항목	내용
DB	PostgreSQL + PGVector 확장
테이블	`langchain_pg_embedding`
컬럼	`document` (청크 텍스트), `embedding` (1024차원 벡터), `metadata` (source_id)
인덱싱	코사인 유사도 기반 벡터 검색

7.2 검색 방식

단계	방식	설명
1차	벡터 검색	쿼리 임베딩과 코사인 유사도 상위 10건
2차	Reranker	`bge-reranker-v2-m3` Cross-Encoder로 재정렬 → 상위 3건

8. 데이터 현황

8.1 전처리 산출물

source.raw → 크롤링 원본 마크다운

source.refined → vLLM 정제 결과

source.summary → 페이지 요약

PGVector → 청크 텍스트 + 임베딩 벡터 + 메타데이터

9. 향후 계획

- **IR 평가 (evaluation_ir):** `testset.json` 기반 Hit Rate, MRR, NDCG 측정 후 MLflow로 실험 기록/비교를 진행할 예정입니다.
- **최적 전략 확정:** 청킹/임베딩/리트리버 분석 및 IR 평가 결과를 종합하여 서비스 적용 전략을 최종 결정할 계획입니다.
- **서비스 반영:** 확정된 청킹 전략과 파라미터를 `chunk/service.py`에 적용할 예정입니다.